

Attention Is All You Need — 논문 정리노트

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin (Google Brain / Google Research / Univ. of Toronto)
arXiv:1706.03762 · NIPS 2017
KUBIG 2026 Summer DL Session 6주차 과제

0. 요약 (TL;DR)

항목	내용
해결하려는 문제	RNN 계열의 순차적 계산(sequential computation) 이 학습 병렬화를 원천 봉쇄하고, 먼 위치 간 의존성 학습을 어렵게 만드는 문제
핵심 아이디어	recurrence와 convolution을 완전히 제거 하고, 오직 attention만으로 encoder-decoder를 구성
구현 수단	Scaled Dot-Product Attention + Multi-Head Attention + Positional Encoding + Position-wise FFN, 전부 residual + LayerNorm으로 감쌘
검증 태스크	WMT 2014 EN-DE / EN-FR 기계번역, 추가로 English constituency parsing (Penn Treebank WSJ)
대표 성과	EN-DE 28.4 BLEU (기존 앙상블 포함 SOTA 대비 +2.0 이상), EN-FR 41.8 BLEU (단일 모델 SOTA)
학습 비용	big 모델 기준 8× P100으로 3.5일 , base 모델은 12시간 (100K step). EN-FR 기준 기존 SOTA의 1/4 이하 FLOPs
이론적 근거	Table 1 — self-attention은 sequential operation O(1) , maximum path length O(1)
논문의 성격	"새 이론"이라기보다, 기존 부품(attention, residual, LayerNorm)을 재배열해 순차성이라는 제약 자체를 제거한 아키텍처 논문

1. 이 논문이 중요한 이유

2017년 이전 sequence transduction의 표준은 LSTM/GRU 기반 encoder-decoder였고, Bahdanau attention은 그 위에 얹는 **보조 장치**였다. 이 논문의 도발은 제목 그대로 — 보조 장치인 줄 알았던 attention이 사실 **본체**였고, RNN 쪽을 통째로 들어내도 성능이 오히려 올라간다는 것.

중요한 건 BLEU 점수 자체가 아니라, 이 구조 변경이 **"모델을 키우는 일"의 성격을 바꿔놨다**는 점이다. RNN은 시퀀스 길이 n 에 대해 $O(n)$ 번의 순차 연산을 강제하므로 GPU를 아무리 많이 붙여도 한 문장 안에서는 병렬화가 안 된다. Transformer는 이 사슬을 끊어, 계산량을 늘리는 것이 곧 **속도 손해 없이** 모델을 키우는 것과 같아졌다. 오늘날의 LLM 스케일링은 사실상 이 한 가지 성질 위에 서 있다.

또 하나 짚을 만한 건, 이 논문이 **5주차 ResNet의 직계 후손**이라는 점이다. 각 sub-layer가 $\text{LayerNorm}(x + \text{Sublayer}(x))$ 형태라는 건 Transformer의 몸통이 통째로 residual stream이라는 뜻이다. ResNet이 없었으면 6층 인코더-디코더 스택도 학습이 안정적으로 되지 않았을 가능성이 크다.

2. 문제 정의: 왜 RNN을 버려야 했나

2-1. 첫 번째 제약 — 순차적 계산 (병렬화 불가)

RNN은 hidden state를 $h_t = f(h_{t-1}, x_t)$ 로 계산한다. h_t 를 구하려면 h_{t-1} 이 먼저 있어야 하므로, 하나의 학습 예제 안에서는 병렬화가 불가능하다. 논문은 이게 시퀀스가 길어질수록 치명적이라고 지적한다. 길이가 길면 메모리 제약 때문에 배치 크기도 못 늘려서, "예제 간 병렬화"라는 도피처마저 막히기 때문이다.

당시에도 factorization trick이나 conditional computation으로 효율을 끌어올린 연구가 있었지만, 논문의 표현대로 순차적 계산이라는 근본 제약 자체는 그대로 남아 있었다. 이 문장이 논문 전체의 출발점이다.

2-2. 두 번째 제약 — 먼 위치 간 경로 길이

장거리 의존성 학습의 난이도는 신호가 forward/backward로 통과해야 하는 경로 길이에 좌우된다. 경로가 짧을수록 배우기 쉽다.

- RNN: 위치 i 와 j 를 잇는 경로 길이 = $O(n)$
- ConvS2S (contiguous kernel): $O(n/k)$, 즉 거리에 선형
- ByteNet (dilated conv): $O(\log_k n)$, 거리에 로그
- Self-Attention: $O(1)$ — 모든 위치가 단 한 번의 연산으로 직결

여기서 중요한 관점 전환이 있다. 이 논문은 "vanishing gradient를 고쳤다"고 말하지 않는다. 대신 정보가 지나가야 할 거리를 아예 없애버렸다고 말한다. 문제를 푼 게 아니라 문제가 성립하는 조건을 제거한 쪽에 가깝다.

2-3. 세 번째 제약 — Seq2Seq의 context vector 병목 (강의자료 보강)

논문 본문보다 강의자료가 더 직관적으로 짚는 지점이다. 기존 seq2seq은 소스 문장 전체를 고정 크기 벡터 v 하나에 압축한다. 문장이 길어지면 앞부분 정보가 손실되는 병목이 생긴다. Bahdanau attention은 디코더가 인코더의 모든 출력을 매 스텝 참조하게 해서 이 병목을 풀었지만, 인코더 자체는 여전히 RNN이라 2-1, 2-2 제약은 살아 있었다.

즉 이 논문의 위치는 이렇게 정리된다: attention은 이미 병목을 푸는 데 성공했다. 그렇다면 병목을 만든 쪽 (RNN)을 왜 아직 남겨두는가?

2-4. 대가는 없었나 — 저자들이 스스로 인정한 트레이드오프

논문은 §2에서 self-attention의 대가도 명시한다. attention은 결국 여러 위치를 가중 평균하는 연산이므로, 정보가 섞이면서 effective resolution이 떨어진다. 그리고 이 손실을 보상하는 장치로 Multi-Head Attention을 도입한다고 밝힌다.

Multi-Head가 "여러 관점을 본다"는 설명으로만 소개되는 경우가 많은데, 논문 §2의 서술을 따르면 **평균화로 인한 해상도 손실을 되찾기 위한 보상 장치**라는 동기가 더 앞선다. 이 순서를 알아두면 Table 3 (A)의 결과가 훨씬 잘 읽힌다.

3. 핵심 아이디어: Attention만으로 짜는 구조

3-1. Attention의 일반 정의

attention은 **query** 하나와 **key-value** 쌍의 집합을 출력 하나로 사상하는 함수다. 출력은 value들의 가중합이고, 각 value의 가중치는 query와 대응 key의 **compatibility function**으로 정해진다.

이 정의가 일반적이라는 게 핵심이다. Q, K, V를 어디서 가져오느냐만 바꾸면 self-attention도 되고 cross-attention도 된다 (→ 3-4).

3-2. Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

왜 dot-product인가: 대안인 additive attention(작은 hidden layer를 가진 FFN으로 compatibility 계산)과 이론적 복잡도는 비슷하지만, dot-product는 **고도로 최적화된 행렬곱 코드로 구현 가능**해서 실제로 훨씬 빠르고 메모리 효율적이다. 이론이 아니라 하드웨어를 보고 내린 선택이다.

왜 $\sqrt{d_k}$ 로 나누는가: d_k 가 작을 때는 두 방식이 비슷하지만, d_k 가 커지면 스케일링 없는 dot-product가 additive보다 나빠진다. 논문의 각주 4가 그 이유를 계산해 보여준다 — q, k의 각 성분이 평균 0, 분산 1인 독립 확률변수라면, 내적 $q \cdot k = \sum q_i k_i$ 는 평균 0, **분산 d_k** 를 갖는다. 즉 $d_k=64$ 면 표준편차가 8이 되어 값이 크게 벌어지고, softmax가 극단적으로 뾰족한 영역(한 원소만 1에 가깝고 나머지는 0)으로 밀려나 **gradient가 거의 0이 된다**. $\sqrt{d_k}$ 로 나누면 분산이 1로 되돌아온다.

이건 "그냥 크니까 나눈다"가 아니라 **분산을 1로 되돌리는 정규화**다. 왜 d_k 가 아니라 $\sqrt{d_k}$ 인지가 여기서 바로 나온다 — 분산이 d_k 라면 표준편차는 $\sqrt{d_k}$ 이므로.

3-3. Multi-Head Attention

$$\begin{aligned} \text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \\ \text{where } \text{head}_i &= \text{Attention}(Q W_i^Q, K W_i^K, V W_i^V) \end{aligned}$$

d_{model} 차원으로 attention을 **한 번** 하는 대신, 학습된 서로 다른 선형 투영으로 h 번 나눠 병렬 수행하고 concat 후 다시 투영한다.

논문 설정: $h = 8$, $d_k = d_v = d_{\text{model}} / h = 512 / 8 = 64$.

연산량이 공짜인 이유: 각 head의 차원이 $1/h$ 로 줄기 때문에, h 개를 다 합쳐도 **full-dimension single-head와 총 연산량이 비슷하다**. "여러 관점"을 추가 비용 없이 얻는 구조다.

논문이 직접 밝히는 동기: *단일 head로는 averaging이 이것(서로 다른 표현 부분공간의 정보에 동시에 주목하는 것)을 방해한다*. — 2-4에서 말한 해상도 손실 보상과 정확히 같은 이야기다.

3-4. Attention이 쓰이는 세 자리 — 같은 함수, 다른 Q/K/V

Transformer 안에서 multi-head attention은 동일한 함수인데 입력 출처만 바꿔 세 군데에 쓰인다. 이 표를 이해하면 구조의 절반이 끝난다.

위치	Query 출처	Key / Value 출처	역할
Encoder self-attention	이전 인코더 layer 출력	동일	입력 문장 내 모든 위치가 서로를 참조
Decoder masked self-attention	이전 디코더 layer 출력	동일	자기 위치 이하 만 참조 (미래 차단)
Encoder-Decoder attention	이전 디코더 layer 출력	인코더 스택 최종 출력	디코더의 각 위치가 입력 전체를 참조 (기존 seq2seq attention과 대응)

3-5. Masking — "미래를 보지 않는다"의 실제 구현

디코더는 auto-regressive이므로 위치 i 의 예측은 i 보다 앞선 출력에만 의존해야 한다. 구현은 의외로 단순하다: **scaled dot-product attention** 내부에서, **softmax의 입력 중 불법 연결에 해당하는 값을 $-\infty$ 로 설정한다.** softmax를 통과하면 그 자리는 정확히 0이 되어 가중치가 사라진다.

여기에 **출력 임베딩이 한 위치 오프셋되어 있다**는 조건이 결합되어야 auto-regressive 성질이 보장된다. 논문은 이 둘을 항상 같이 언급한다.

왜 마스킹 자리에 0이 아니라 $-\infty$ 를 넣는지 헷갈리기 쉬운데, 마스킹은 **softmax 이전에** 적용되기 때문이다. softmax 이후에 0을 곱하면 남은 가중치의 합이 1이 아니게 된다. $-\infty$ 를 넣으면 $\exp(-\infty)=0$ 이 되면서 정규화도 자동으로 남은 위치들 사이에서만 이뤄진다.

3-6. Position-wise Feed-Forward Network

$$\text{FFN}(x) = \max(0, x \cdot W_1 + b_1) \cdot W_2 + b_2$$

각 위치에 **독립적·동일하게** 적용되는 2층 MLP. `d_model = 512 → d_ff = 2048 → 512`.

두 가지 뉘앙스: - 선형변환은 위치마다 동일하지만 **layer마다**는 다른 **파라미터**를 쓴다. - 논문은 이를 "**kernel size 1인 convolution 두 개**" 로도 볼 수 있다고 표현한다. 위치 간 정보 교환이 전혀 없다는 걸 강조하는 표현이다.

역할 분담이 깔끔하다. **attention**은 위치 간 정보를 섞고(**mixing across positions**), **FFN**은 각 위치 내부에서 **표현을 변환한다(transform per position)**. Transformer의 한 layer는 이 두 연산의 교대다.

3-7. Positional Encoding — 순서 정보를 되돌려 놓기

recurrence도 convolution도 없으므로, 모델은 기본적으로 **입력을 순서 없는 집합으로 본다**. 그래서 위치 정보를 명시적으로 주입해야 한다.

```
PE(pos, 2i) = sin( pos / 10000^(2i / d_model) )
PE(pos, 2i+1) = cos( pos / 10000^(2i / d_model) )
```

임베딩과 같은 차원(d_{model})이라 그냥 **더한다**. 파장은 2π 부터 $10000 \cdot 2\pi$ 까지 등비수열을 이룬다.

왜 하필 sinusoid인가 — 논문이 제시하는 이유는 두 가지다.

1. 임의의 고정 오프셋 k 에 대해 $PE_{\{pos+k\}}$ 가 $PE_{\{pos\}}$ 의 **선형 함수로 표현될 수 있다**. 그래서 모델이 상대적 위치로 attend하는 법을 쉽게 배울 것이라고 가설을 세웠다.
2. 학습된 positional embedding과 달리, **학습 때 본 적 없는 더 긴 시퀀스로 외삽(extrapolate)** 할 수 있을 것으로 기대했다.

정직한 부분: 저자들은 학습된 positional embedding도 실험했고, **결과가 거의 동일했다고** Table 3 (E)에서 밝힌다(PPL 4.92 vs 4.92, BLEU 25.8 vs 25.7). 즉 sinusoid를 고른 진짜 이유는 성능이 아니라 **외삽 가능성이라는 가설** 때문이다. 그리고 그 가설을 이 논문에서 검증하지는 않았다 (→ 9-2).

3-8. 임베딩과 Softmax의 가중치 공유

- 입력 임베딩, 출력 임베딩, pre-softmax 선형변환 **세 곳이 같은 가중치 행렬을 공유한다**.
- 임베딩 layer에서는 그 가중치에 $\sqrt{d_{\text{model}}}$ 을 곱한다.

$\sqrt{d_{\text{model}}}$ 을 곱하는 이유는 논문에 명시되어 있지 않다. 통상적인 해석은 임베딩 값의 스케일을 positional encoding(진폭이 대략 1)과 맞춰주기 위한 것이다. PE를 그냥 더하는 구조이므로, 임베딩이 너무 작으면 위치 정보가 의미 정보를 덮어버린다. 논문이 설명하지 않는 구현 디테일이라는 점은 알아둘 만하다.

4. 아키텍처 정리

4-1. Encoder

- $N = 6$ 개의 동일한 layer 스택
- 각 layer는 2개 sub-layer: ① multi-head **self**-attention ② position-wise FFN
- 각 sub-layer는 $\text{LayerNorm}(x + \text{Sublayer}(x))$ 로 감싼다 (residual → LayerNorm 순, 즉 **Post-LN**)
- residual 덧셈이 가능하려면 차원이 맞아야 하므로, **임베딩 포함 모든 sub-layer 출력이 $d_{\text{model}} = 512$**

4-2. Decoder

- 역시 $N = 6$ 개 layer
- 각 layer는 3개 sub-layer: ① **masked** multi-head self-attention ② encoder-decoder attention ③ FFN
- 동일하게 residual + LayerNorm

4-3. 하이퍼파라미터 한눈에 (base / big)

항목	base	big
N (layer 수)	6	6
d_model	512	1024
d_ff	2048	4096
h (head 수)	8	16
d_k = d_v	64	64
P_drop	0.1	0.3 (EN-FR은 0.1)
ϵ_{ls} (label smoothing)	0.1	0.1
train steps	100K (12시간)	300K (3.5일)
파라미터 수	65M	213M

5. Why Self-Attention — Table 1의 논증

논문 §4는 "성능이 좋았다"가 아니라 세 가지 기준을 먼저 세우고 layer 타입을 비교하는 방식으로 논증한다. 이 절이 논문에서 가장 잘 쓰인 부분이라고 생각한다.

세 가지 desiderata: ① layer당 총 계산 복잡도 ② 병렬화 가능한 계산량(= 최소 순차 연산 수) ③ 장거리 의존성의 최대 경로 길이

Layer Type	Complexity per Layer	Sequential Ops	Max Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k n)$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

여기서 반드시 짚어야 할 조건: self-attention이 recurrent보다 빠른 건 $n < d$ 일 때다. $O(n^2 \cdot d)$ vs $O(n \cdot d^2)$ 를 비교하면 바로 나온다. 논문은 word-piece/BPE 기반 문장 표현에서는 보통 $n < d(=512)$ 라서 이 조건이 성립한다고 주장한다.

이 조건절이 이 논문의 유효 범위를 규정한다. 문장 단위 번역에서는 성립하지만, 문서 단위나 긴 문맥으로 가면 n 이 d 를 넘어서면서 $O(n^2)$ 이 그대로 비용으로 돌아온다. 이후 등장한 Longformer, Linformer, FlashAttention 계열이 전부 이 지점을 공략한 연구다. 논문 스스로도 restricted self-attention(이웃 크기 r 로 제한, 경로 길이는 $O(n/r)$ 로 증가)을 future work로 언급한다.

부수적 이점: attention 분포를 들여다보면 개별 head가 서로 다른 역할을 학습하고, 상당수는 문장의 **구문적·의미적 구조와 관련된 행동**을 보인다. 즉 해석 가능성(interpretability)이 덤으로 따라온다.

6. 학습 세팅 (Implementation)

6-1. 데이터

- **EN-DE**: WMT 2014, 약 450만 문장쌍, BPE, 소스-타겟 **공유** vocabulary 약 37,000 토큰
- **EN-FR**: WMT 2014, 3,600만 문장, word-piece vocabulary 32,000
- 배치: 비슷한 길이끼리 묶고, 배치당 소스/타겟 각각 약 **25,000 토큰**

6-2. 하드웨어

- 8× NVIDIA **P100** 단일 머신
- base: step당 0.4초 × 100K step = 12시간 / big: step당 1.0초 × 300K step = 3.5일

6-3. Optimizer — warmup + inverse sqrt decay

Adam ($\beta_1=0.9$, $\beta_2=0.98$, $\epsilon=10^{-9}$)에, 다음 learning rate 스케줄을 쓴다.

```
lr_rate = d_model^(-0.5) * min( step^(-0.5), step * warmup^(-1.5) )
warmup_steps = 4000
```

- 처음 4000 step: lr을 **선형 증가**
- 이후: step 수의 **역제곱근에 비례해 감소**

이 스케줄은 옵션이 아니라 사실상 필수 부품이다. Post-LN 구조에서는 초기 step에 gradient가 크게 튀어 warmup 없이는 학습이 발산하기 쉽다는 것이 이후 널리 알려졌다. 논문은 스케줄을 제시만 하고 **왜 필요한지는 설명하지 않는다** — 재현할 때 가장 자주 걸려 넘어지는 지점이다.

6-4. Regularization 세 가지

1. **Residual Dropout**: 각 sub-layer 출력에, **residual 덧셈과 정규화 이전에** 적용. 추가로 encoder/decoder 양쪽에서 **임베딩 + positional encoding의 합에도** 적용. base는 $P_{drop} = 0.1$
2. **Label Smoothing** ($\epsilon_{ls} = 0.1$): 논문이 정직하게 밝히길, 모델이 더 "불확실"해지도록 학습되므로 **perplexity는 나빠진다**. 그런데도 **accuracy와 BLEU는 좋아진다**
3. (넓게 보면) checkpoint averaging — base는 마지막 5개, big은 마지막 20개 체크포인트 평균

Label smoothing의 $PPL \downarrow$ / $BLEU \uparrow$ 는 **평가 지표가 서로 다른 것을 측정한다는** 걸 보여주는 좋은 사례다. PPL은 모델이 정답에 얼마나 확신하는지를, BLEU는 실제 생성 품질을 본다. 확신을 꺾으면 전자는 나빠지고

후자는 좋아질 수 있다.

6-5. 추론 세팅

- beam search, **beam size 4**, length penalty $\alpha = 0.6$
- 최대 출력 길이 = 입력 길이 + 50, 가능하면 조기 종료

7. 실험 결과

7-1. 기계번역 (Table 2)

Model	EN-DE BLEU	EN-FR BLEU	Training FLOPs (EN-DE)
ByteNet	23.75	—	—
GNMT + RL	24.6	39.92	$2.3 \cdot 10^{19}$
ConvS2S	25.16	40.46	$9.6 \cdot 10^{18}$
MoE	26.03	40.56	$2.0 \cdot 10^{19}$
GNMT + RL Ensemble	26.30	41.16	$1.8 \cdot 10^{20}$
ConvS2S Ensemble	26.36	41.29	$7.7 \cdot 10^{19}$
Transformer (base)	27.3	38.1	$3.3 \cdot 10^{18}$
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$

읽어낼 점 세 가지:

1. **base 모델(65M)이 기존 앙상블을 전부 이긴다** — 그것도 ConvS2S 앙상블 대비 약 1/23의 FLOPs로. 성능보다 이 효율 격차가 더 충격적이다.
2. big의 EN-DE 28.4는 기존 최고(앙상블 포함)보다 **2.0 BLEU 이상** 높다.
3. EN-FR에서 big은 기존 SOTA 단일 모델을 **1/4 미만의 학습 비용**으로 넘어선다.

7-2. Ablation (Table 3) — 각 부품이 정말 필요한가

newstest2013 dev set 기준, base(PPL 4.92 / BLEU 25.8 / 65M)를 기준으로.

행	변형	결과 (BLEU)	해석
(A)	h=1 (single head)	24.9	최적 대비 0.9 BLEU 하락 → multi-head는 실제로 기여한다
(A)	h=16 / h=32	25.8 / 25.4	head가 너무 많아도 나빠진다 → head당 차원이 지나치게 작아짐
(B)	d_k 축소 (64→16)	25.1	key 차원을 줄이면 품질 하락
(C)	N=2 / d_model=256	23.7 / 24.5	예상대로 작은 모델은 나쁘다
(C)	d_model=1024 / d_ff=4096	26.0 / 26.2	크면 좋다
(D)	P_drop = 0.0	24.6	dropout 없으면 overfit — 1.2 BLEU 손해
(D)	$\epsilon_{ls} = 0.0$	25.3	label smoothing도 기여
(E)	learned positional embedding	25.7	sinusoid와 사실상 동일

(A)와 (B)의 조합이 이 표에서 가장 흥미롭다. head를 늘리는 것(A)과 d_k를 줄이는 것(B)은 사실 같은 축의 양면이다 — $d_k = d_{\text{model}}/h$ 이므로 h를 늘리면 d_k가 줄어든다. h=32에서 성능이 떨어지는 건 head 개수가 문제가 아니라 **d_k=16이 너무 작아서**라고 읽는 게 자연스럽다.

그리고 논문 스스로 (B)에 대해 이렇게 적는다: *compatibility를 판단하는 일이 쉽지 않으며, dot product보다 정교한 compatibility 함수가 이로울 수 있다.* — 자기 설계의 핵심 부품에 대해 개선 여지를 명시한 드문 서술이다.

7-3. 일반화 검증 — English Constituency Parsing (Table 4)

번역 전용 구조가 아님을 보이려는 검증. Penn Treebank WSJ 약 40K 문장, 4-layer / d_model=1024.

이 태스크가 어려운 이유: 출력이 **강한 구조적 제약**을 받고, 입력보다 **길이가 훨씬 길다**. 게다가 RNN seq2seq은 소량 데이터 환경에서 SOTA에 도달한 적이 없었다.

Parser	Training	WSJ 23 F1
Vinyals & Kaiser et al. (2014)	WSJ only	88.3
Petrov et al. (2006) — BerkeleyParser	WSJ only	90.4
Dyer et al. (2016) — RNNG	WSJ only	91.7
Transformer (4 layers)	WSJ only	91.3
Transformer (4 layers)	semi-supervised	92.7
Dyer et al. (2016)	generative	93.3

태스크 전용 튜닝을 거의 하지 않고도(dropout, lr, beam size만 조정) WSJ-only 40K에서 **BerkeleyParser**를 넘어선다. 다만 RNNG(91.7 / generative 93.3)는 넘지 못했고, 논문은 이것을 숨기지 않고 명시한다.

8. 논문의 기여 정리

1. recurrence와 convolution 없이 attention만으로 sequence transduction이 가능함을 최초로 입증했다.

2. Scaled Dot-Product Attention과 그 스케일링의 **정량적 근거(분산 d_k)** 를 제시했다.
3. Multi-Head Attention으로 **추가 연산 비용 없이** 다중 표현 부분공간을 학습하는 방법을 제시했다.
4. Table 1을 통해 layer 타입을 **복잡도·병렬성·경로 길이 3축으로 비교하는 틀**을 세웠다. 성능 이전에 구조적 논거를 제시한 것.
5. 번역 SOTA를 **훨씬 적은 학습 비용으로** 달성했다 — 성능과 효율을 동시에.
6. parsing 실험으로 **태스크 일반성**을 보였다.
7. warmup + inverse-sqrt lr 스케줄, label smoothing, checkpoint averaging 등 **재현 가능한 학습 레시피**를 함께 공개했다 (코드: tensor2tensor).

9. 한계 및 비판적 검토

9-1. $O(n^2)$ 메모리·연산이라는 근본 비용

Table 1은 self-attention을 유리하게 보이게 하지만 그건 $n < d$ 조건 하에서다. 문서 요약, 긴 문맥 처리처럼 n 이 커지는 순간 $O(n^2 \cdot d)$ 가 그대로 병목이 된다. 논문도 restricted attention을 future work로 남겼을 뿐 해결하지 않았다.

9-2. sinusoidal PE의 외삽 능력을 검증하지 않았다

sinusoid를 선택한 명시적 이유가 "학습보다 긴 시퀀스로 외삽할 수 있을지도 모른다"인데, **그 실험이 논문에 없다**. Table 3 (E)가 보여준 건 "학습된 임베딩과 성능이 같다"뿐이다. 이후 연구에서 sinusoidal PE의 길이 외삽 성능이 기대만큼 좋지 않다는 것이 밝혀졌고, RoPE·ALiBi 같은 상대 위치 인코딩이 이 자리를 대체했다.

9-3. 숫자 불일치 — EN-FR BLEU 41.8 vs 41.0

초록과 Table 2는 EN-FR big 모델을 **41.8**로 적는데, §6.1 본문은 같은 모델을 **41.0**으로 서술한다. arXiv 개정 과정에서 남은 불일치로 보이며(현재 v7), 인용 시 Table 2 기준 **41.8**이 통용된다. 논문을 실제로 읽었다면 걸리는 지점이라 기록해 둔다.

9-4. 학습 안정성에 대한 설명 부재

warmup 4000 step이 왜 필요한지 논문은 설명하지 않는다. Post-LN 구조가 초기 학습에서 불안정하다는 점은 이후 Pre-LN 연구(Xiong et al., 2020)에서 규명됐고, 오늘날 대부분의 구현은 **Pre-LN**($x + \text{Sublayer}(\text{LayerNorm}(x))$)을 쓴다. 즉 **논문 원본 구조를 그대로 쓰는 코드는 지금 거의 없다**.

9-5. 검증 태스크의 좁은 범위

번역 2개 + parsing 1개가 전부다. "attention만으로 충분하다"는 제목의 주장 범위에 비하면 근거가 좁고, 이 논문 시점에서 지금과 같은 범용성이 입증된 것은 전혀 아니다. 결론에서 저자들도 이미지·오디오·비디오로의 확장을 **계획(plan)**으로만 적는다.

9-6. 해석 가능성 주장의 강도

"head가 구문적·의미적 구조를 학습하는 듯 보인다"는 서술은 부록의 시각화 몇 개에 근거한 **정성적 관찰**이다. 정량적 분석은 없다. 이후 attention weight를 설명(explanation)으로 읽는 것이 타당한지에 대한 반론 (*Attention is not Explanation*, 2019)이 나왔다는 점도 같이 알아둘 필요가 있다.

10. 이후 연구가 채운 공백

논문의 미해결 지점	이후 전개
$O(n^2)$ 비용	Sparse / Longformer / Linformer / Performer, FlashAttention (IO 최적화로 정확도 손실 없이 완화)
PE의 길이 외삽	Relative PE (Shaw et al. 2018), RoPE , ALiBi
Post-LN 불안정성	Pre-LN , RMSNorm, DeepNet
encoder-decoder 형태	BERT (encoder-only), GPT (decoder-only)로 분화
번역 전용 검증	ViT (이미지), Whisper(오디오), DETR(객체 탐지) — 강의자료 p48~50이 다루는 바로 그 확산
FFN 비용	Mixture-of-Experts로 sparse 확장
head의 역할	head pruning 연구 — 상당수 head는 제거해도 무방하다는 결과

11. 구현 관점 체크포인트 (재현 시 자주 틀리는 곳)

- $\sqrt{d_k}$ 스케일링 위치 — softmax **이전**, dot product **직후**. 순서를 바꾸면 의미가 없어진다.
- 마스킹은 **softmax 이전에** $-\infty$ (실무에선 $-1e9$). softmax 이후 0을 곱하면 정규화가 깨진다.
- residual 덧셈 이전에 dropout** — `LayerNorm(x + Dropout(Sublayer(x)))` 순서.
- 임베딩 + PE 합에도 dropout 적용**. 빠뜨리기 쉬운 항목이다.
- 임베딩 가중치에 $\sqrt{d_{\text{model}}}$ 곱하기** — 이걸 빼면 PE가 의미 정보를 압도한다.
- warmup을 반드시 구현**. Post-LN을 쓰면서 warmup을 빼면 발산 가능성이 높다.
- 디코더 입력은 한 위치 **shift** (`<sos>` 추가). 마스킹만으로는 auto-regressive가 보장되지 않는다.
- Adam $\beta_2 = 0.98$** — 기본값 0.999가 아니다.
- 모든 sub-layer 출력 차원 = d_{model}** — residual 덧셈의 전제 조건.
- 평가 시 checkpoint averaging** 유무로 BLEU가 눈에 띄게 달라진다. 비교할 때 조건을 맞출 것.

12. 정리하며

이 논문을 5주차 ResNet과 이어서 읽으면 구도가 선명해진다. ResNet은 "**깊게 쌓을 수 있는가**"라는 질문에 shortcut connection으로 답했고, Transformer는 "**넓게 병렬화할 수 있는가**"라는 질문에 recurrence 제거로

답했다. 그리고 Transformer는 그 답을 ResNet이 만들어 준 residual stream 위에 올려놓았다. 두 논문 모두 새로운 수학이 아니라 **정보가 흐르는 경로를 다시 설계**해서 문제를 풀었다는 점이 공통점이다.

한 가지 더. 제목은 "Attention Is All You Need"지만, ablation(Table 3)이 실제로 보여주는 건 dropout 없으면 1.2 BLEU가 날아가고, label smoothing도 필요하고, warmup 없으면 학습이 어렵다는 사실이다.

attention만으로 되는 게 아니라 **attention을 중심에 두고 나머지를 잘 붙이면 된다는** 쪽이 정확하다. 제목의 단호함과 본문의 정직함 사이의 간극이, 오히려 이 논문을 좋은 논문으로 만든다고 생각한다.